

1. **3 levels**
3 branches
Node values stored in branches: **2, 3, 5**
3 leaves
Node values stored in leaves: **1, 4, 6**
Root node value: **3**
Node 2's sibling node value: **5**
Node 2's children node values: **4, 6**
2. Pre-order: **Kirk, Chekov, Spock, Scotty, McCoy, Khaaaaan!, Sulu, Uhuru**
In-order: **Chekov, Kirk, Khaaaaan!, McCoy, Scotty, Spock, Sulu, Uhuru**
Post-order: **Chekov, Khaaaaan!, Sulu, McCoy, Scotty, Uhuru, Spock, Kirk**
3. Pre-order: **Lisa, Bart, Homer, Flanders, Marge, Maggie, Milhouse, Smithers**
In-order: **Bart, Flanders, Homer, Lisa, Maggie, Milhouse, Marge, Smithers**
Post-order: **Flanders, Homer, Bart, Milhouse, Maggie, Smithers, Marge, Lisa**
- 4.

a.

```
// 1. get size
Windsurf: Refactor | Explain | X
public int size() { //
    return size(overallRoot);
}

Windsurf: Refactor | Explain | Generate Javadoc | X
private int size(IntTreeNode current) { // helper method, does actual recursive work
    if (current == null) { // base case
        return 0;
    }

    // current node + nodes in left subtree + nodes in right subtree
    return 1 + size(current.left) + size(current.right); // recursive case
}

3 public class IntTreeMain {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | X
4     public static void main(String[] args) {
5         IntTree tree = new IntTree(s: "[3 [5 [1] null ] [2 [4] [6]]]");
6         System.out.println(tree);
7         System.out.println(tree.size());
8     }
9 }
10
```

PROBLEMS 173 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS GITLENS

```
duey@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Internet\ Plug-Ins/JavaAppletPlugin
cp /var/folders/w3/0x9k0j1s65gczs9x9ztngbw0000gn/T/cp_d9w8q6i0cjxyrjpj7td6i724a.jar we
overallRoot
[3]
  [5] [2]
   [1] [4] [6]
```

b.

```
// 2. return number of empty branches (null links/childrens)
Windsurf: Refactor | Explain | X
public int countEmpty() {
    return countEmpty(overallRoot);
}

Windsurf: Refactor | Explain | Generate Javadoc | X
private int countEmpty(IntTreeNode current) {
    if (current == null) { // base case
        return 1; // empty subtree counts as 1 empty branch
    }

    // empty branches in left subtree + empty branches in right subtree
    return countEmpty(current.left) + countEmpty(current.right); // recursive case
}

3 public class IntTreeMain {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | X
4     public static void main(String[] args) {
5         IntTree tree = new IntTree(s: "[3 [5 [1] null ] [2 [4] [6]]]");
6         System.out.println(tree);
7         // System.out.println(tree.size());
8         System.out.println(tree.countEmpty());
9     }
10 }

PROBLEMS 173 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS GITLENS
duey@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Internet\ Plug-Ins/JavaAppletPlugin
cp /var/folders/w3/0x9k0j1s65gczs9x9ztngbw0000gn/T/cp_d9w8q6i0cjxyrpj7td6i724a.jar we
overallRoot
[3]
 / \
[5] [2]
 / \ / \
[1] [4] [6]
```

c.

```
public int depthSum() {
    return depthSum(overallRoot, weight: 1); // root has weight 1
}

Windsurf: Refactor | Explain | Generate Javadoc | X
private int depthSum(IntTreeNode current, int weight) {
    if (current == null) { // base case
        return 0; // empty subtree counts as 0
    }

    // current node sum (weight * data) + sum of left subtree + sum of right subtree
    return weight * current.data
        + depthSum(current.left, weight + 1) // increment weight as we go down
        + depthSum(current.right, weight + 1);
}

5 IntTree tree = new IntTree(s: "[3 [5 [1] null ] [2 [4] [6]]]");
6 System.out.println(tree);
7 // System.out.println(tree.size());
8 // System.out.println(tree.countEmpty());
9 System.out.println(tree.depthSum());
10 }

PROBLEMS 173 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS GITLENS
duey@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Internet\ Plug-Ins/JavaAppletPlug
cp /var/folders/w3/0x9k0j1s65gczs9x9ztngbw0000gn/T/cp_d9w8q6i0cjxyrpj7td6i724a.jar
overallRoot
[3]
 / \
[5] [2]
 / \ / \
[1] [4] [6]
50
```

d.

```
public void printLeaves() {  
    if (overallRoot == null) { // base case  
        System.out.println("no leaves");  
    }  
    else {  
        System.out.print("leaves:");  
        printLeaves(overallRoot);  
        System.out.println();  
    }  
}
```

Windsurf: Refactor | Explain | Generate Javadoc | X

```
private void printLeaves(IntTreeNode current) {  
    if (current == null) { // base case  
        return;  
    }  
  
    // if current node has no children (leaf), print it  
    if (current.left == null && current.right == null) {  
        System.out.print(" " + current.data);  
    }  
    else {  
        // recurse down right then left (reverse order)  
        printLeaves(current.right);  
        printLeaves(current.left);  
    }  
}
```

10



tree.printLeaves();|

PROBLEMS 173 OUTPUT DEBUG CONSOLE

```
duey@MacBook-Air-Duy CS211 % /usr/
cp /var/folders/w3/0x9k0j1s65gczs9x
overallRoot
      [3]
     /  \
    [5]  [2]
   /  \ /  \
  [1] [4] [4] [6]
leaves: 6 4 1
```